Brief Agent - Manual Loop Architecture Specification

Executive Summary

This document defines the architectural specification and implementation guide for the **Brief Agent - Manual Loop** workflow. Built inside n8n using OpenRouter and Google Gemini Flash, this pattern explicitly orchestrates multi-step AI agent interactions through granular, low-level nodes (Set, Switch, Basic LLM Chain, Structured Output Parser, Airtable Search, Aggregate, and If nodes) instead of relying on the high-level n8n AI Agent node abstraction.

While production environments prefer the native n8n AI Agent node for maintainability and compact reasoning logs, this explicit manual orchestration pattern provides full visibility into every loop iteration, state accumulation step, tool dispatch logic, and safety gate execution.

Architectural Comparison

Dimension	AI Agent Node (High-Level)	Manual Loop Architecture (Low-Level)
Node Count & Complexity	Minimal (~10 nodes). Abstracted execution loop.	High (~25 nodes). Fully explicit loop mechanics.
Loop Visibility	Encapsulated. State accumulation is hidden.	Transparent. Full visibility into history payloads and tool routing.
Maintenance Overhead	Low. Standardized schema updates and compact logs.	Higher. Direct maintenance of state, schemas, and routing nodes required.
Custom Control	Constrained to agent node framework capabilities.	Fine-grained. Supports arbitrary routing logic and deterministic safety gates.

6-Phase Execution Loop

Every iteration of the manual multi-step agent consists of six distinct phases:

Phase 1: Read Input & State

Set up and pass the system prompt, initial user request, accumulated tool results, and state object into the model turn (Prepare Model Turn).

Phase 2: Call LLM

Send the conversation history and available tool catalog to Google Gemini Flash via OpenRouter. Receive either a structured `tool_call` or a `final` brief output.

Phase 3: Parse & Branch on Action Type

Utilize a Structured Output Parser to extract $json.output. A Switch node routes output 0 to tool execution (tool_call) and output 1 to the completion formatter (`final`), with an explicit fallback for malformed model outputs.

Phase 4: Tool Execution & Dispatch

Route tool requests to designated Airtable nodes across five Southern Cross database tables:

- `lookupAccount` → Accounts table (`{companyName} = "{{ $json.output.toolInput.companyName }}"`)
- `getUsageMetrics` → Usage Metrics table (`{accountId} = "{{ $json.output.toolInput.accountId }}"`)
- `getTicketHistory` → Tickets table (`{accountId} = "{{ $json.output.toolInput.accountId }}"`)
- `getBillingStatus` → Billing table (`{accountId} = "{{ $json.output.toolInput.accountId }}"`)
- `getRecentInteractions` → Interactions table (`{accountId} = "{{ $json.output.toolInput.accountId }}"`)

Phase 5: Aggregate & Append Result

Pass each Airtable Search result through an Aggregate node to collapse multiple record items into a single `records` array, preventing parallel workflow branching. Update `messages.history` and increment `iterationCount`.

Phase 6: Check Stop & Loop Back

An If node evaluates whether `iterationCount` is below the safety threshold (max cap = 5 iterations). If true, it loops back to phase 1 (Prepare Model Turn); if false, it routes to Force Stop with an incomplete status note.

Step-by-Step n8n Workflow Construction Guide

1. Workflow Shell & State Initialization

- **Manual Trigger:** Initiates execution.
- **Test Request (Set Node):** Mode set to *Manual Mapping*. Defines field `request` = `"Brief me on Acme Corp for tomorrow's renewal call."`
- **Init History (Set Node):** Sets initial state object containing system instructions, user request string, empty `history` array, and `iterationCount = 0`.
- **Prepare Model Turn (Set Node):** Combines system context, user request, and conversation history into a unified model prompt payload for subsequent iterations.

2. Model Turn & Output Parsing

- **Call Model (Basic LLM Chain Node):** Connected to an **OpenRouter Chat Model** running `google/gemini-2.5-flash-lite` (or closest Gemini Flash model) at low temperature (`0.2`).
- **Structured Output Parser:** Enforces a JSON schema requiring the model to return either a `tool_call` object (with `toolName` and `toolInput`) or a `final` output object containing the complete structured brief.
- **Action Type (Switch Node):** Evaluates `$json.output.actionType`:
    - Output `0`: `tool_call` → Proceed to tool dispatch.
    - Output `1`: `final` → Route to `Format Final Brief`.
    - Fallback: Route to `Invalid Model Response` node (`outcome = "model_error"`).

3. Tool Dispatch & Airtable Integration

- **Dispatch Tool (Switch Node):** Evaluates `$json.output.toolName` across five routes:
    - Route `0`: `lookupAccount` → Searches `Accounts` table in Southern Cross base.
    - Route `1`: `getUsageMetrics` → Searches `Usage Metrics` table.
    - Route `2`: `getTicketHistory` → Searches `Tickets` table.
    - Route `3`: `getBillingStatus` → Searches `Billing` table.
    - Route `4`: `getRecentInteractions` → Searches `Interactions` table.
- **Aggregate Nodes (1 per tool):** Configured with *Aggregate: All Item Data Into a Single List → Put Output in Field: records*. Ensures each tool step outputs exactly one item containing an array, preventing history path duplication.

4. State Update & Loop Control

- **Update History (Set Node):** Receives single-item outputs from all five Aggregate nodes. Appends the executed tool call and returned data array to `messages.history`, and increments `iterationCount`.
- **Continue or Stop? (If Node):** Evaluates `$json.iterationCount` < `5`:
    - **TRUE:** Loops output back to `Prepare Model Turn`.
    - **FALSE:** Routes to `Force Stop` node (`outcome = "incomplete"`, note = `"The agent reached the maximum number of tool iterations before producing a final brief."`).

5. Success Path & Final Formatting

- **Format Final Brief (Set Node):** Connected from `Action Type` output 1 (`final`). Emits final payload with `outcome = "brief"` and the structured customer brief object.

Architectural Key Takeaways

1. **The Loop is the Architecture:** Regardless of whether implemented implicitly via the AI Agent node or explicitly via manual loops, the fundamental pattern remains: *Send Context → Receive Decision → Execute Tool → Append Result → Repeat*.
2. **Hard Safety Caps Belong in Orchestration:** Prompt instructions ("stop after 5 tool calls") serve as guidance, but deterministic loop checks at the orchestrator level provide the true security safety net.

3. **Data Aggregation is Mandatory:** Unaggregated search results branching into state update nodes multiply execution paths exponentially. Aggregating output lists maintains deterministic, single-stream conversation state.